Lab 21: Collaborative AI Coding: Pair Programming and Version Control Integration

Name: D Rishwanth
Rollno: 2503A52395
Batch: 14

Task 1: AI-Assisted Pair Programming

Question Requirements:

Task 1 – AI-Assisted Pair Programming

Use AI to generate an initial program (e.g., a simple To-Do List Manager).

Ask AI to suggest enhancements (such as error handling, additional features, or refactoring).

Compare the original and improved versions to understand the role of AI in collaboration.

Initial Program:

```
tasks = []

def add_task(text):
    tasks.append(text)


add_task("Finish lab")

print(tasks)
```

Improved Program:

```
class TodoList:
    """Store tasks and track completion."""
    def __init__(self):
        self.items = []


    def add(self, title):
        """Add a non-empty task and return its one-based number."""
        title = title.strip()
        if not title:
            raise ValueError("Task title cannot be empty.")
```

```
        self.items.append({"title": title, "done": False})
        return len(self.items)

    def complete(self, number):
        """Mark a valid one-based task number complete."""
        if not isinstance(number, int) or not 1 <= number <= len(self.items):
            raise IndexError("Task number does not exist.")
        self.items[number - 1]["done"] = True

    def display(self):
        """Return tasks with a readable completion marker."""
        return [f"[{'x' if item['done'] else ' '}] {item['title']}" for item in self.items]

manager = TodoList()
manager.add("Finish lab")
manager.complete(1)
print(manager.display())
assert manager.display() == ["[x] Finish lab"]
```

Comparison and Reflection:

The improved class validates task names and indexes, stores completion state, and formats output. AI helped draft the structure; the programmer remains responsible for reviewing behavior and maintaining the code.

Example Output:

Initial: ['Finish lab']
Improved: ['[x] Finish lab']

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]

Task 2: Version Control Integration

Question Requirements:

Task 2 – Version Control Integration

Initialize a Git repository for your project.

Perform the following steps:

Create and switch to a new branch.

Add AI-assisted modifications.

Commit with a meaningful message.

Merge changes back into the main branch.

Reflect on how version control supports collaborative development with AI assistance.

Version Control and Collaboration Steps:

Example workflow commands (run inside the project directory):

$ git init

$ git branch -M main

$ git add .

$ git commit -m "Add initial project"

$ git switch -c feature/ai-improvements

Make and review the AI-assisted change

$ git add .

$ git commit -m "Add validated AI-assisted improvements"

$ git switch main

$ git merge --no-ff feature/ai-improvements

$ git log --oneline --graph --all

Reflection:

Branches isolate changes for review, commits preserve snapshots, and the merge makes the reviewed change part of main. Commands are an example procedure; they do not claim a repository was created for this report.

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]

Task 3: AI for Commit Message Generation

Question Requirements:

Task 3 – AI for Commit Message Generation

Use AI to generate concise, meaningful commit messages based on the changes made.

Compare AI-generated messages with human-written ones.

Discuss how AI can improve documentation and traceability in Git history.

Change Summary and Message Comparison:

Change summary: add input validation and completion tracking to the To-Do List Manager.

AI suggestion: "Add validation and task completion tracking"

Human message: "Improve todo handling"

AI-Generated and Human-Written Messages:

Change: reject blank task names, validate task numbers, and track completed tasks.

AI-generated commit message:

feat(todo): validate tasks and track completion

Human-written commit message:

Improve todo handling

Discussion:

An AI suggestion can improve consistency and clarify history when it accurately describes the diff. A developer should compare the message with the actual changes before committing; a message must not claim work that was not done.

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]

Task 4: Pair Programming Simulation

Question Requirements:

Task 4 – Pair Programming Simulation

Student A writes the initial version of a program.

Student B, with AI assistance, suggests improvements or adds features.

Both students collaborate using Git branching and merging to integrate their work.

Student A Initial Version:

```
# Student A: initial version
tasks = []
def add_task(title):
    tasks.append(title)
```

Student B AI-Assisted Improvement:

```
# Student B: reviewed improvement
class TodoList:
    """A minimal task list with validation and completion state."""
    def __init__(self):
        self.tasks = []

    def add(self, title):
        """Add a non-empty task."""
        if not isinstance(title, str) or not title.strip():
            raise ValueError("A task title is required.")
        self.tasks.append({"title": title.strip(), "done": False})

    def complete(self, index):
        """Complete a zero-based task index."""
        if not 0 <= index < len(self.tasks):
            raise IndexError("Task index is out of range.")
        self.tasks[index]["done"] = True
```

Suggested collaboration flow:

Student A commits the base; Student B creates a feature branch,

reviews/tests the addition, commits it, then merges through review.

Branch and Merge Example:

git switch -c feature/student-b-improvements

git add todo.py

git commit -m "Add validated task completion"

git switch main

git merge --no-ff feature/student-b-improvements

Collaboration Reflection:

Pair programming combines an initial implementation with a second review and a focused enhancement. Git history identifies the base and feature commits, making discussion and rollback easier.

Example Result:

Student A supplies basic storage; Student B adds checks and completion state. The feature is reviewed before merge.

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]

Task 5: AI-Assisted Debugging

Question Requirements:

Task 5 – AI-Assisted Debugging

• Create a program with logical or syntax errors.

• Use AI to identify and fix the errors.

• Compare the buggy and corrected versions.

Buggy Version:

Buggy version: loop skips the final element and has an invalid endpoint

def average(values):

```
    return sum(values) / (len(values) - 1)


# average([2, 4, 6]) incorrectly returns 6.0
```

Corrected Version:

```
# Corrected version
def average(values):
    """Return the arithmetic mean of a non-empty numeric sequence."""
    if not values:
        raise ValueError("values must not be empty")
    return sum(values) / len(values)


assert average([2, 4, 6]) == 4
try:
    average([])
except ValueError:
    pass
else:
    raise AssertionError("Empty input should be rejected")
print(average([2, 4, 6]))
```

Expected Output:

```
4.0
```

Debugging Explanation:

The denominator must be the number of values, not one less. An empty-list guard avoids division by zero. The assertions check the corrected result and boundary behavior.

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]

Task 6: Feature Expansion Using AI

Question Requirements:

Task 6 – Feature Expansion using AI

• Develop a basic application (e.g., Calculator).

• Use AI to add advanced features (history, validation).

• Analyze improvements.

Basic Calculator:

```
def calculate(a, operator, b):
    if operator == "+": return a + b
    if operator == "-": return a - b
    if operator == "*": return a * b
    if operator == "/": return a / b
```

Expanded Calculator with History and Validation:

```
class Calculator:
    """Calculate basic arithmetic and retain successful operations."""
    def __init__(self):
        self.history = []

    def calculate(self, first, operator, second):
        """Validate inputs, calculate, and record the expression."""
        if not isinstance(first, (int, float)) or isinstance(first, bool):
            raise TypeError("First value must be numeric.")
        if not isinstance(second, (int, float)) or isinstance(second, bool):
            raise TypeError("Second value must be numeric.")
        operations = {"+": lambda: first + second, "-": lambda: first - second,
                      "*": lambda: first * second, "/": lambda: first / second}
        if operator not in operations:
            raise ValueError("Use +, -, *, or /.")
        if operator == "/" and second == 0:
```

```
            raise ZeroDivisionError("Cannot divide by zero.")
        result = operations[operator]()
        self.history.append(f"{first} {operator} {second} = {result}")
        return result


calc = Calculator()
assert calc.calculate(8, "/", 2) == 4
assert calc.calculate(3, "+", 5) == 8
print(calc.history)
```

Example Output:

```
['8 / 2 = 4.0', '3 + 5 = 8']
```

Feature Analysis:

The expanded calculator rejects unsupported operators and invalid numeric inputs, checks division by zero, and stores successful calculations in history. Its tests cover division and addition; additional tests can cover invalid input paths.

Output Screenshot (paste evidence here):

[Leave this area for the actual execution or output screenshot.]